

partnr Dashboard

Skill
Graph

Profile

GPP

Job
Applications

Maneesh Koti B... ▾

Build a Twitter Sentiment Analysis API with BERT Fine-Tuning

[Back](#)

Mandatory

Domain

Machine Learning

Skills

API Development

Artificial Intelligence

Machine Learning

Natural Language Processing

Public Relations

transfer learning

User Acceptance Testing

Difficulty

Intermediate

Tools

Docker

FastAPI

Gradio

Hugging Face

Pandas

Python

Pytorch

Streamlit

Tensorflow

Industries

E-commerce

**Pending Submission**

TIME REMAINING

Please submit your work when ready.

3dDeadline: **15 Aug 2026 at 04:59 PM**

Overview

Instructions

Resources

Submit

Objective

You will build a complete sentiment analysis system using a pre-trained BERT model. You will learn to implement a full MLOps pipeline, including data preprocessing, model fine-tuning with the Hugging Face library, performance evaluation, and deployment as a REST API. This project is highly relevant for real-world applications like social media monitoring, customer feedback analysis,

and brand management.

Description

Background

Sentiment analysis is a core task in Natural Language Processing (NLP) that involves determining the emotional tone behind a body of text. Modern NLP heavily relies on Transformer architectures, with models like BERT (Bidirectional Encoder Representations from Transformers) setting the standard for performance on various language tasks. Instead of training these massive models from scratch, the common practice is to use pre-trained models and fine-tune them on a specific downstream task, like sentiment classification. This process, known as transfer learning, significantly reduces training time and data requirements while achieving state-of-the-art results.

This project will guide you through building a production-style sentiment analysis service. You will use the Hugging Face ecosystem, a de-facto standard for working with Transformer models. The system will be containerized using Docker, ensuring reproducibility and ease of deployment. You'll also implement an API to serve predictions and a simple web interface for interaction, mimicking how ML models are integrated into larger applications. Finally, you will incorporate experiment tracking, a crucial practice in MLOps for logging and comparing model performance.

Implementation Details

Follow these steps to build, train, and deploy your sentiment analysis system. The project should be fully containerized and launchable with a single command.

Step 1: Project Setup

Create a clear directory structure for your project. Here is a recommended layout:

```
# Project Root
├── data/
│   ├── raw/           # Place initial dataset here
│   └── processed/     # Store cleaned data here
├── model_output/      # Store fine-tuned model artifacts
├── results/           # Store evaluation metrics and predictions
├── scripts/           # For preprocessing, training, and prediction scripts
│   ├── preprocess.py
│   └── train.py
```

```

├── batch_predict.py
├── src/
│   ├── api.py          # FastAPI application
│   └── ui.py           # Streamlit/Gradio application
├── tests/              # Optional but recommended: tests for your API
├── Dockerfile.api
├── Dockerfile.ui
├── docker-compose.yml
├── .env.example
└── README.md

```

Step 2: Data Preprocessing

You will need a dataset for sentiment analysis. The `imdb` dataset from Hugging Face Datasets is a good choice. Create a script (`scripts/preprocess.py` , see `data-preprocessing-script`) that loads the raw data, performs necessary cleaning (e.g., removing special characters, URLs), and splits it into training and testing sets, saving them to `data/processed/` .

Step 3: Model Fine-Tuning

Create a training script (`scripts/train.py` , see `model-training-script`) that:

1. Loads the pre-trained BERT model and tokenizer from Hugging Face (e.g., `bert-base-uncased`).
2. Loads your preprocessed data.
3. Fine-tunes the model on the training data.
4. Evaluates the model on the test data and saves the metrics to `results/metrics.json` (see `model-evaluation-metrics`).
5. Integrates with an experiment tracker by logging hyperparameters and final metrics to `results/run_summary.json` (see `experiment-tracking-output`).
6. Saves the fine-tuned model artifacts (model weights, config, tokenizer files) to the `model_output/` directory (see `model-artifacts-persistence`).

Step 4: API Development

Build a REST API using FastAPI (`src/api.py`) that serves the fine-tuned model. It must have two endpoints:

1. A health check endpoint (`GET /health`) (see `api-health-check`).
2. A prediction endpoint (`POST /predict`) that takes a text string and returns the sentiment and confidence score (see `api-sentiment-prediction`).

The API should load the model artifacts from the `model_output/` directory during startup.

Step 5: Web Interface

Create a simple web interface using Streamlit or Gradio (`src/ui.py`) that allows a user to enter text and see the sentiment prediction returned by the API. This UI should make a request to your FastAPI backend.

Step 6: Containerization

Create `Dockerfile` s for both the API (`Dockerfile.api`) and the UI (`Dockerfile.ui`). Then, create a `docker-compose.yml` file to orchestrate both services. The `docker-compose.yml` must include health checks to ensure services are running correctly. The API service should be built with the fine-tuned model artifacts from `model_output/` copied into the image.

Step 7: Batch Prediction

Implement a script (`scripts/batch_predict.py` , see `batch-prediction-script`) that can take a CSV file of texts and generate a new CSV file containing the original text along with predicted sentiments and confidence scores.

Step 8: Documentation

Create a `.env.example` file listing all necessary environment variables (see `env-example-file`). Also, write a comprehensive `README.md` that explains the project, how to set it up, and how to use the API and scripts.

Final Submission Artifacts Checklist

Your final submission should be a Git repository containing the following:

- ☐ `README.md` : A detailed project overview, setup instructions, and API usage guide.
- ☐ `docker-compose.yml` : Main file to run all services.
- ☐ `Dockerfile.api` : Dockerfile for the prediction API.
- ☐ `Dockerfile.ui` : Dockerfile for the web interface.
- ☐ `.env.example` : Template for all required environment variables.
- ☐ Application Code:
 - ☐ `scripts/preprocess.py`
 - ☐ `scripts/train.py`
 - ☐ `scripts/batch_predict.py`
 - ☐ `src/api.py`
 - ☐ `src/ui.py`

- ☐ `model_output/` : This directory containing the final model should be part of the API's Docker image, but can be gitignored if the files are large. The README should explain how to generate it.
- ☐ `results/` : This directory can also be gitignored, but the README should explain how scripts generate files here.
- ☐ `tests/` : (Optional) A directory with tests for your application logic.

Implementation Guidelines

1. **Model Choice:** While the task specifies BERT, feel free to experiment with smaller variants like DistilBERT for faster inference, or other models like RoBERTa. Document your choice and rationale in the README.
2. **Error Handling:** Implement robust error handling in your API. What happens if the input text is empty or invalid? Return appropriate HTTP status codes and error messages.
3. **Configuration Management:** Use environment variables for configurations like model paths, port numbers, etc. Avoid hardcoding values in your scripts.
4. **Code Quality:** Write clean, modular, and well-commented code. Separate concerns where possible (e.g., data loading logic, model logic, API logic).
5. **Performance:** For the API, consider the trade-off between model size/accuracy and inference speed. Loading the model into memory only once when the API starts is critical for performance.
6. **Security:** While not a primary focus, be mindful of basic security practices. For example, ensure any external libraries are from trusted sources.

FAQ

1. Q: Which dataset should I use?

1. A: The `imdb` dataset from the Hugging Face Datasets library is an excellent choice. It is well-structured for binary sentiment classification. You can adapt the model for multi-class (positive, negative, neutral) classification by modifying the final layer.

2. Q: My model files are too large to commit to Git. What should I do?

1. A: Use Git LFS or simply add the `model_output/` directory to your `.gitignore`. Your `README.md` must provide clear instructions on how to run the `scripts/train.py` script to generate the necessary model artifacts before building the Docker containers.

3. Q: How do I ensure the API service waits for the model to be trained?

1. A: The training process should be a separate, one-time step performed before deploying the API. The `Dockerfile.api` should copy the pre-trained artifacts from `model_output/` into the image. It should not run the training script.

4. Q: Can I use a different framework than FastAPI or Streamlit?

1. A: Yes, you can use alternatives like Flask for the API or Gradio for the UI. However, your solution must meet all the functional requirements specified, including being fully containerized.

5. Q: What should the health check endpoint (`api-health-check`) do?

1. A: It should simply return a 200 OK status with a small JSON payload like `{"status": "ok"}` to indicate that the API server is running and responsive.

6. Q: How should the batch prediction script (`batch-prediction-script`) be run?

1. A: It should be a standalone script that can be executed from the command line within the container or locally (if the environment is set up). For example: `python scripts/batch_predict.py --input-file data/unseen/data.csv --output-file results/predictions.csv`.

Core Requirements

1. The project must be fully containerized using Docker and orchestrated with a `docker-compose.yml` file at the root of the repository. Running `docker-compose up` must start all required services (API, UI) without errors.

File Location: `docker-compose.yml` (repository root)

Required Services:

1. `api` : The FastAPI backend service.
2. `ui` : The Streamlit/Gradio frontend service.

Requirements:

1. The `api` service must build from a `Dockerfile.api`.
2. The `ui` service must build from a `Dockerfile.ui`.
3. Both services must define a `healthcheck` to verify they are running correctly.
4. All necessary ports must be exposed and mapped.

Verification:

1. Run `docker-compose up --build -d` from the project root.
2. Verify that two containers (`api` and `ui`) start successfully.
3. Run `docker ps` and check that both containers report a 'healthy' status within 3 minutes.

2. A `.env.example` file must be present in the repository root, documenting all environment variables required for the application to run.

File Location: `.env.example` (repository root)

Required Content:

1. All environment variables needed for the API and UI services (e.g., `API_PORT`, `MODEL_PATH`).
2. Use placeholder values, not real secrets.

Format Example:

```
# Port for the API service
API_PORT=8000
# Path to the model artifacts inside the container
MODEL_PATH=/app/model_output
```

Verification:

1. Check for the existence of the `.env.example` file at the repository root.
2. Verify the file contains key-value pairs for configuration.

3. A script must be provided to preprocess the raw dataset. It should handle cleaning, tokenization preparations, and splitting the data into training and testing sets.

File Location: `scripts/preprocess.py`

Functionality:

1. The script should be runnable from the command line.
2. It should take a raw data source (e.g., from Hugging Face Datasets) and output processed files.
3. It must create `train.csv` and `test.csv` files in the `data/processed/` directory.

Output File Schema (`train.csv` , `test.csv`):

1. Must be a CSV file with at least two columns: `text` and `label` .

Verification:

1. A file must exist at `scripts/preprocess.py` .
2. After running the script, verify that `data/processed/train.csv` and `data/processed/test.csv` are created.
3. Verify the CSV files contain the `text` and `label` columns.

4. A script must be provided to fine-tune the BERT model. This script should handle loading data, training, evaluation, and saving the final model artifacts.

File Location: `scripts/train.py`

Functionality:

1. Loads preprocessed data from `data/processed/train.csv` and `data/processed/test.csv`.
2. Fine-tunes a BERT-like model.
3. Saves the final fine-tuned model artifacts to the `model_output/` directory.
4. Generates an evaluation metrics file at `results/metrics.json`.

Verification:

1. A file must exist at `scripts/train.py`.
2. After running the script, the `model_output/` directory must be created and contain model files.
3. After running the script, the `results/metrics.json` file must be created and contain valid evaluation data.

5. The fine-tuned model artifacts must be saved to a designated directory for the API to use.

File Location: `model_output/`

Required Files:

1. `config.json` (model configuration)
2. `pytorch_model.bin` or `tf_model.h5` (model weights)
3. `tokenizer_config.json` (tokenizer configuration)
4. `vocab.txt` (or similar vocabulary file)

Verification:

1. After running the training script, check for the existence and non-empty status of the required files in the `model_output/` directory.
2. The `Dockerfile.api` must copy the contents of `model_output/` into the final image.

6. The training script must produce a JSON file containing key evaluation metrics for the fine-tuned model.

File Location: `results/metrics.json`

Required Schema:

```
{
  "accuracy": "float (0.0 to 1.0)",
  "precision": "float (0.0 to 1.0)",
  "recall": "float (0.0 to 1.0)",
  "f1_score": "float (0.0 to 1.0)"
}
```

Verification:

1. Check for the existence of `results/metrics.json` after the training script completes.
2. Parse the JSON file and verify that it contains the four required keys.
3. Verify that the value for each key is a floating-point number between 0 and 1.

7. The training script must generate a summary file that logs key hyperparameters and final metrics, simulating an experiment tracking run.

File Location: `results/run_summary.json`

Required Schema:

```
{
  "hyperparameters": {
    "model_name": "string",
    "learning_rate": "float",
    "batch_size": "integer",
    "num_epochs": "integer"
  },
  "final_metrics": {
    "accuracy": "float",
    "f1_score": "float"
  }
}
```

Verification:

1. Check for the existence of `results/run_summary.json` after the training script completes.
2. Parse the JSON file and verify its structure matches the required schema.
3. Check that all specified fields are present with the correct data types.

8. The API must provide a health check endpoint to confirm it is operational.

Endpoint: `GET /health`

Response (200 OK):

```
{  
  "status": "ok"  
}
```

Verification:

1. Start the services using `docker-compose up`.
2. Send a `GET` request to the `/health` endpoint on the exposed API port.
3. Verify the HTTP status code is 200.
4. Verify the response body is a JSON object matching `{"status": "ok"}`.

9. The API must provide an endpoint to perform sentiment analysis on a single piece of text.

Endpoint: POST /predict

Request Body Schema:

```
{
  "text": "string"
}
```

Response Body Schema (200 OK):

```
{
  "sentiment": "string (e.g., 'positive', 'negative', 'neutral')",
  "confidence": "float (0.0 to 1.0)"
}
```

Verification:

1. Send a POST request to /predict with a valid JSON body, e.g.,
{"text": "I love this product!"}.
2. Verify the HTTP status code is 200.
3. Verify the response body contains the sentiment (string) and confidence (float) keys.
4. Verify the sentiment value is one of the expected classes.
5. Verify the confidence value is between 0.0 and 1.0.

10. A script must be provided to run predictions on a batch of texts from a CSV file.

File Location: `scripts/batch_predict.py`

Input File:

1. Path provided as an argument (e.g., `data/unseen/predict_data.csv`).
2. Format: CSV file with a header and a column named `text` .

Output File:

1. Path provided as an argument (e.g., `results/predictions.csv`).
2. Format: CSV file with the original columns plus `predicted_sentiment` and `confidence` .

Verification:

1. A file must exist at `scripts/batch_predict.py` .
2. Create a sample input CSV at `data/unseen/predict_data.csv` .
3. Run the script.
4. Verify that the output file `results/predictions.csv` is created.
5. Verify the output CSV contains the `predicted_sentiment` and `confidence` columns with valid data.

11. A simple UI application must be provided and configured to run as a service.

File Location: `src/ui.py`

Configuration:

1. The `docker-compose.yml` file must define a service named `ui`.
2. The `ui` service must expose a port for the UI (e.g., 8501 for Streamlit).
3. The `ui` service must build from `Dockerfile.ui`.

Verification:

1. Check for the existence of `src/ui.py`.
2. Parse `docker-compose.yml` and verify the existence of the `ui` service.
3. Verify the `ui` service definition includes a `ports` mapping.

partnr

[About Us](#) [Contact Us](#) [Privacy Policy](#)

[Terms and Conditions](#)

All rights reserved. Copyright, Partnr 2025-26



Report Issue